

Project 2 — LLM / RAG · Arabic Legal Document Q&A

Arabic legal documents · Embeddings + Vector DB · vLLM · RAGAS · Langfuse · Guardrails

The business problem. A law firm or government authority in the Arab world needs a system that answers questions about Arabic legal documents — contracts, Egyptian Civil Code articles, Saudi regulations — accurately and with citations. Hallucinations are legally unacceptable.

Why this project for this course RAGAS evaluation is genuinely hard · no GPU required to start, an API-based LLM works fine · embedding drift is real as legal language shifts across jurisdictions · Langfuse tracing reveals the full RAG chain · and AWQ quantization makes the LLM run fully offline.

Best if you have no GPU — the track is focused on evaluation and observability.

YOUR INPUT DATA — PROJECT 2

The corpus is the Egyptian Civil Code (القانون المدني المصري, Law of 1948) — a bilingual Arabic/English PDF, article by article.

This PDF is not your corpus. It is your raw input Your first deliverable on this track is a script that converts it into **structured JSON**, committed to the repo and tracked with DVC. Do not embed raw `pdftotext` output — a RAG system built on unstructured page text cannot cite an article number, and an answer a lawyer cannot verify is worthless.

STEP 0 FOR PROJECT 2 — BUILD THE CORPUS

1. Target schema. One record per article. Start here and extend if your ingestion needs it:

```
{
  "article_number": 147,
  "book": "Obligations or Personal Rights",
  "chapter": "Sources of Obligations",
  "section": "Contracts",
  "topic": "The Effects of a Contract",
  "text_ar": "... العقد شريعة المتعاقدين، فلا يجوز نقضه ولا تعديله",
  "text_en": "The contract makes the law of the parties. It can be revoked ...",
  "is_repealed": false,
  "source_page": 34,
  "citation": "Egyptian Civil Code, Article 147"
}
```

2. What will actually break during extraction— budget an evening for this, and write what you hit in your report:

- **The bilingual two-column layout interleaves** Raw extraction mixes Arabic and English mid-sentence. You need column-aware extraction or a positional heuristic, not a naive text dump.

- **Arabic is right-to-left** and extractors frequently return reversed or disconnected glyphs. Normalize with a library that handles reshaping, and **eyeball 20 random articles** before you trust the whole file.
- **Article numbers are in Arabic-Indic numerals** (١٤٧). Normalize them to Western digits — `article_number` must be an integer you can sort, range-query and cite.
- **Some articles are repealed** (Articles 54–80 were repealed by Presidential Decree). Flag them rather than deleting them: **"that article no longer exists" is a correct and valuable answer**, and a system that silently omits them will hallucinate around the gap.
- **Headings carry the hierarchy** — Book, Chapter, Section, and the topic labels. Capture them as metadata during the pass; they become your retrieval filters and they are far cheaper than reparsing later.
- **Arabic diacritics, hamza forms and alef variants** need normalizing consistently, or the same word will embed as two different tokens.

3. Validate before you embed. Write a test asserting: article numbers are contiguous with no unexplained gaps, every record has non-empty `text_ar`, no record exceeds a sane length (a giant record means a failed split), and repealed articles are flagged. **A silent parsing bug becomes a hallucination three steps later, and by then you will blame the model.**

4. Chunk by article, not by token count. A legal article is a self-contained unit of meaning with a natural citation. Fixed 512-token windows cut mid-provision and destroy the citation. Long articles with numbered paragraphs may split by paragraph while keeping the article number on every chunk.

Chunking strategy is exactly the MLflow experiment your Module 2 checklist asks for — log `chunk_size`, `overlap`, `embedding_model` and RAGAS faithfulness per run, and let the numbers decide.

5. Make it reproducible. The conversion script is a `dvc.yaml` stage, and both the PDF and the resulting JSON are DVC-tracked, so `dvc repro` rebuilds the corpus from the source PDF and your reviewer can regenerate it exactly.

6. Cite by article number. Your `/ask` response returns `sources` as article citations, not chunk IDs or page numbers. That is how a lawyer verifies an answer, and it is also what makes RAGAS **context precision** and **faithfulness** meaningful for you rather than abstract.

TWO THINGS THIS CORPUS GIVES YOU FOR FREE

A bilingual evaluation set. Every article exists in Arabic and English, so you can ask the same question in both languages and check that retrieval returns the same article — a real, cheap test of whether your embedding model actually handles Arabic, and a strong ablation for your Module 5 scorecard.

And a real embedding-drift scenario: a corpus of contract law queried about family law is a genuine semantic shift, which is exactly what your MMD and domain-classifier detectors from Mini Project 5 are for.

What you ship, module by module

OUTPUT – PROJECT 1 · DEEP LEARNING

Package + API	AraBERT wrapper. <code>{ "المنتج رائع" } → {"label":"positive", "confidence":0.94}</code>
Track + Version	MLflow: 5+ runs. DVC: labeled reviews. GitHub Actions CI on push.
Serve	BentoML micro-batching. TensorRT FP16. Locust: p95 before vs after.
Optimize	Distil 12→6 layers. INT8 ONNX. Benchmark: accuracy × latency × size.
Monitor	Evidently PSI on vocabulary. Prometheus + Grafana. Alert: PSI > 0.25.

OUTPUT – PROJECT 2 · LLM / RAG

Package + API	<code>src/rag.py</code> (ingest + query). <code>{ "ما هي شروط العقد؟" } → {"answer":..., "sources":[...]}</code>
Track + Version	MLflow: chunking experiments (chunk_size, overlap, RAGAS faithfulness). DVC: document corpus.
Serve	BentoML wraps RAG. vLLM serves the generative model. Streaming <code>/ask</code> .
Optimize	AWQ-4bit generative model. Re-ranker distillation. RAGAS before vs after.
Monitor	RAGAS on ≥ 50 questions. Langfuse traces every request. Cosine drift. Token cost.

ORDERING NOTE

Optimization is delivered in **Week 5** and Observability in **Week 6**, so the two rows above are listed in delivery order. If an earlier version of the project guide shows Monitor before Optimize, the deliverables themselves are identical — only the week shifts.

The 10-point rubric

Same criteria for both tracks. Your project certificate and your referral eligibility are decided here.

#	AREA	MODULE	WHAT YOU BUILD	PASS CONDITION
01	Code & packaging	M1	<code>pyproject.toml</code> , <code>src/</code> layout, OOP model class, type hints	<code>pip install -e .</code> works. Model loads without errors.
02	API endpoint	M1	<code>/predict</code> with Pydantic, <code>/health</code> , async handlers	<code>curl /predict</code> returns correct output.
03	Docker	M1	Dockerfile, <code>docker-compose.yml</code> , 3-command README	<code>docker compose up</code> && <code>curl localhost:8000/predict</code> works on the reviewer's machine.
04	MLflow tracking	M2	≥ 5 runs, params + metrics + artifacts, best model in Registry	MLflow UI shows the comparison. Model promoted to Production.
05	DVC data versioning	M2	Dataset tracked, <code>dvc.yaml</code> pipeline, <code>dvc repro</code> works	<code>git checkout + dvc pull + dvc repro</code> reproduces the result.
06	GitHub Actions CI/CD	M2	Lint → test → build → push on every PR. Quality gate.	Merge a PR. Actions green. Docker image pushed to the registry.
07	Production serving	M3	BentoML (DL) or vLLM + BentoML (LLM). Locust tested.	p95 latency documented. Locust report in <code>/reports/</code> .
08	Monitoring	M5	≥ 1 drift metric (DL) or RAGAS faithfulness (LLM). Grafana panel.	Screenshot in README. Alert threshold set.
09	Peer review	Post-course	Written review ≥ 300 words, all 5 areas	Submitted on time. Covers setup, code, strength, 2 improvements, extension.
10	README & architecture	M4/M5	3-command setup, architecture diagram, session changelog	The reviewer runs it without asking you anything.

Completion checklists

PROJECT 1 · DEEP LEARNING – CODE, PACKAGING, TRACKING, VERSIONING

- ☐ `src/` layout with `pyproject.toml` — `pip install -e .` works
- ☐ AraBERT or CAMEL-BERT loaded in a class with type hints
- ☐ FastAPI `/predict: {text: str} → {label, confidence, model_version}`
- ☐ Pydantic rejects empty text — 422 returned, tested with `curl`
- ☐ `/health` returns `{status: healthy}` — used by the Docker healthcheck
- ☐ `docker build -t arabic-sentiment .` passes; `docker compose up` starts the service on port 8000

- ☐ README: exactly 3 commands to run on any machine
- ☐ MLflow tracking server running — UI accessible
- ☐ ≥ 5 runs: `model_name`, `lr`, `batch_size`, `accuracy`, `f1_macro`
- ☐ MLflow comparison screenshot in `/reports/mlflow_comparison.png`
- ☐ Best model registered as `ArabicSentiment` → promoted to Production
- ☐ Labeled dataset tracked with DVC — `dvc status` shows clean
- ☐ `dvc repro` reproduces training with the same metrics (± 0.005)
- ☐ GitHub Actions: lint → test → docker build → push on `main`
- ☐ CI fails if `f1_macro` drops below baseline — documented in README

PROJECT 1 • DEEP LEARNING – SERVING, MONITORING, OPTIMIZATION

- ☐ BentoML service with `batchable=True` Runner — `bentoml models list` shows the model
- ☐ `bentoml serve` runs `/predict` at `localhost:8000`
- ☐ Locust report before optimization in `/reports/locust_fastapi.html`
- ☐ TensorRT FP16 engine built — committed or documented
- ☐ Locust after TensorRT in `/reports/locust_trt.html` — p95 in README
- ☐ Batch scoring: ≥ 1,000 samples, output to `/data/scoring/output/`
- ☐ nginx canary config: weight 5/95 — rollout stages in README
- ☐ Evidently DataDrift report on the latest batch — saved in `/reports/`
- ☐ PSI computed for `text_length` and `confidence_score` features
- ☐ Drift check triggered after every batch scoring run
- ☐ Prometheus exposes the PSI score at `/metrics`
- ☐ Grafana: ≥ 2 panels (p95 + PSI) — screenshot in README
- ☐ Alert: PSI > 0.25 → notification documented in README
- ☐ 12-layer AraBERT distilled to a 6-layer student — model file committed
- ☐ INT8 ONNX quantization applied — `model_int8.onnx` committed
- ☐ Benchmark table: original vs distilled vs INT8 (accuracy, p95, size)
- ☐ Final architecture diagram in README covering all 5 sessions

PROJECT 2 • LLM / RAG – CODE, PACKAGING, TRACKING, VERSIONING

- ☐ `src/` layout with `pyproject.toml` — `pip install -e .` works

- ☐ Egyptian Civil Code PDF converted to **structured JSON**, one record per article, committed to the repo
- ☐ Extraction validated: article numbers normalized to integers, Arabic text spot-checked on 20 articles, repealed articles flagged
- ☐ Ingestion pipeline: JSON → chunk by article → embed → vector store
- ☐ FastAPI `/ask : {question: str} → {answer: str, sources: list[str]}` — **sources are article citations, not chunk IDs**
- ☐ Pydantic rejects an empty question — 422 returned and tested
- ☐ `/health` returns `{status: healthy, documents_indexed: N}`
- ☐ Dockerfile includes the vector store and embedded documents; `docker compose up` starts on port 8000
- ☐ README: 3 commands to run Q&A on any machine
- ☐ MLflow logs each config: `chunk_size`, `overlap`, `embedding_model`, `faithfulness`
- ☐ ≥ 5 runs compared — MLflow screenshot in `/reports/`
- ☐ Best chunking config registered in the MLflow Registry
- ☐ Source PDF and derived JSON both tracked with DVC — `dvc pull` fetches them, `dvc repro` rebuilds the JSON from the PDF
- ☐ `dvc repro` re-indexes the corpus reproducibly from tracked documents
- ☐ GitHub Actions: lint → test → rebuild index → push Docker image
- ☐ CI fails if RAGAS faithfulness < 0.75 on a 20-question test set

PROJECT 2 • LLM / RAG – SERVING, MONITORING, OPTIMIZATION

- ☐ BentoML wraps the RAG pipeline with an async `/ask` endpoint
- ☐ vLLM serves the generative LLM — model name in README
- ☐ Streaming: tokens appear progressively in `curl` output
- ☐ Locust report at 50 concurrent users in `/reports/`
- ☐ Batch re-indexing script tested with at least one new document
- ☐ Canary rollout config documented in README
- ☐ RAGAS on ≥ 50 questions — all 4 metrics logged
- ☐ RAGAS results stored in MLflow — trend visible across sessions
- ☐ Alert: faithfulness < 0.80 → notification triggered
- ☐ Langfuse self-hosted — every `/ask` creates a trace with spans
- ☐ RAGAS faithfulness score attached to each Langfuse trace

- ☐ Cosine similarity on query embeddings vs baseline — drift logged
- ☐ Token usage in Prometheus — Grafana shows cost/hour
- ☐ Guardrails PII detection active on all `/ask` responses
- ☐ Generative model quantized to AWQ-4bit — weights documented
- ☐ RAGAS scores: original vs quantized — ≤ 0.03 faithfulness drop
- ☐ Latency before vs after quantization in README
- ☐ Final architecture diagram covering all 5 session components

•

=